



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES



COMPUTER SCIENCE AND ENGINEERING

BRANCH : CSE
YEAR/SEM : III
SUB CODE & TITLE : CSA 1322 – Theory of Computation
SUBMISSION DATE : 15.07.2026

Activity 2 : NFA Design

Problem 1: Design an NFA for a Given Language

Problem:

Construct an NFA over $\Sigma = \{a, b\}$ that accepts all strings that end with ab .

Student Task:

- Draw the NFA (states, transitions, start & final state).
- Write the transition table.
- Test the NFA for: ab , aab , bab , aba .

Learning Outcome:

Understand how NFAs can “guess” the ending using nondeterminism.

SOLUTION:

Language

$$L = \{w \in \{a, b\}^* \mid w \text{ ends with } ab\}$$

NFA

- States: $Q = \{q_0, q_1, q_2\}$
- Alphabet: $\Sigma = \{a, b\}$
- Start State: q_0
- Final State: q_2

Transition Table

State	a	b
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

Working

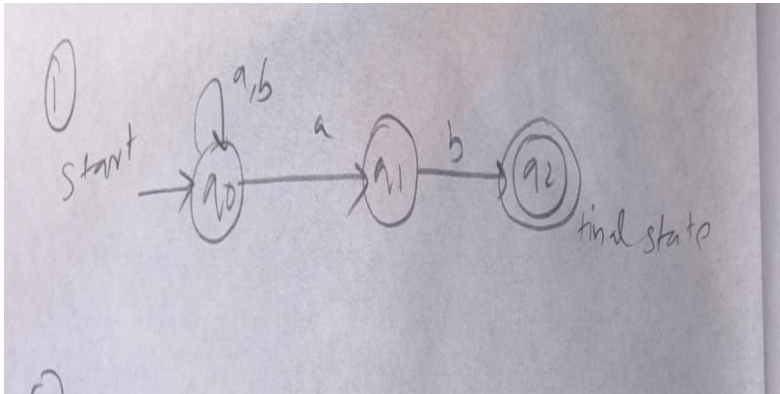
- q_0 keeps reading any symbols.
- Whenever an **a** is found, the NFA can “guess” it may be the start of the ending **ab** and move to q_1 .
- If the next symbol is **b**, it reaches q_2 (accept state).
- Hence only strings ending in **ab** are accepted.

Testing

String	Accepted?	Reason
ab	✓ Yes	Ends with ab
aab	✓ Yes	Ends with ab
bab	✓ Yes	Ends with ab
aba	✗ No	Ends with ba

Learning Outcome

- Understand nondeterminism.
- Learn how an NFA guesses the correct ending.



Problem 2 : Convert Regular Expression to NFA

Problem:

Convert the regular expression

$$(a+b)^*ab$$

into an NFA using Thompson's Construction.

Student Task:

- Construct small NFAs for a, b, +, and *.
- Combine them step by step.
- Mark start and final states clearly.

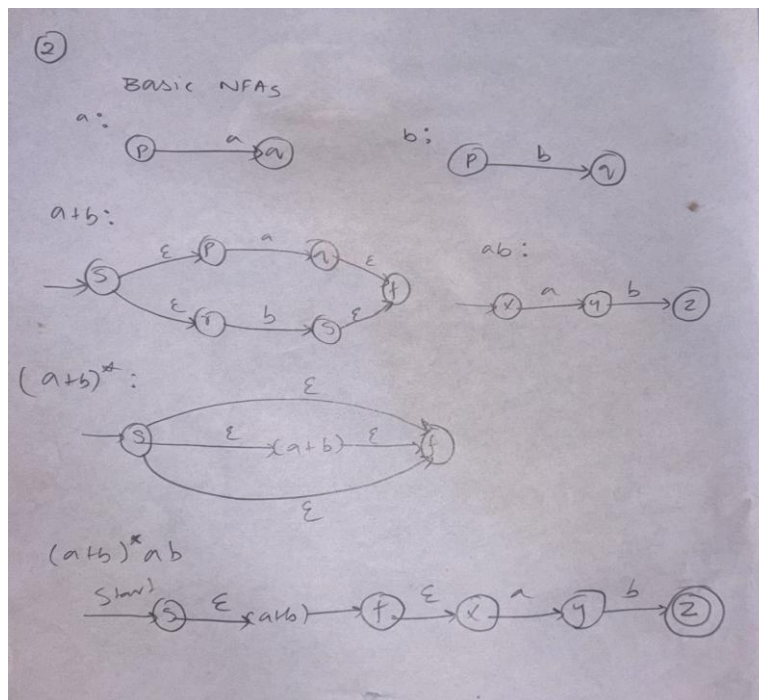
Learning Outcome:

Understand relation between Regular Expression $\leftrightarrow$ NFA.

SOLUTION:

Regular Expression

$$(a+b)^*ab$$



Start State

s

Final State

f

Learning Outcome

- Understand Thompson's Construction.
- Learn conversion from Regular Expression to NFA.

Problem 3: NFA with Constraint (Limited States Challenge)

Problem:

Design an NFA over $\Sigma = \{a, b\}$ that accepts strings containing at least one **a** and at least one **b**, using no more than 3 states.

Student Task:

- Try multiple designs.
- Justify why the design works.

Learning Outcome:

Encourages optimization thinking and creative use of nondeterminism.

SOLUTION:

Language

Strings containing

- at least one **a**
 - at least one **b**
-

States

- q_0 = Start
 - q_1 = Seen **a**
 - q_2 = Seen both **a** and **b** (Final)
-

Transition Table

State	a	b
$\rightarrow q_0$	$\{q_1\}$	$\{q_0\}$
q_1	$\{q_1\}$	$\{q_2\}$
$*q_2$	$\{q_2\}$	$\{q_2\}$

Working

- q_0 waits for first **a**
 - q_1 remembers that **a** has appeared.
 - After reading **b**, move to q_2 .
 - q_2 accepts every remaining symbol.
-

Example

String	Result
--------	--------

String	Result
ab	✓
ba	✗
aaab	✓
bbbb	✗
abab	✓

Why only 3 states?

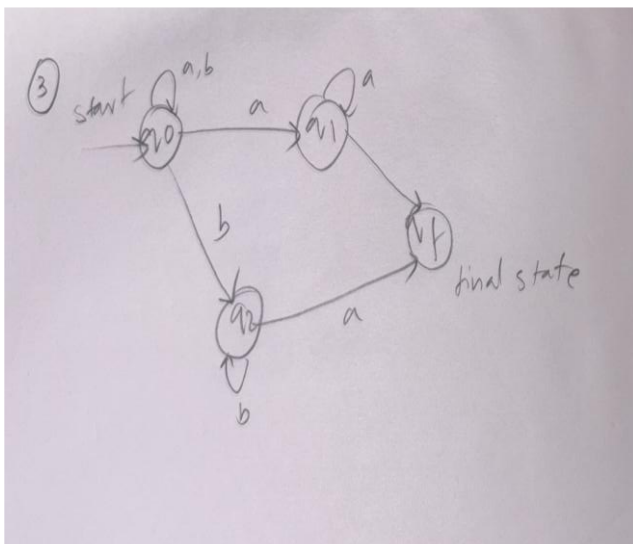
Because we only need to remember

- nothing seen
- only a seen
- both seen

This is the minimum design.

Learning Outcome

- Practice optimization.
- Use nondeterminism efficiently.



Problem 4: NFA for Substring Detection

Problem:

Design an NFA that accepts all strings over $\{0,1\}$ that contain the substring 101.

Student Task:

- Draw the NFA.
- Explain how nondeterminism helps detect the substring.

- Test with: 101, 01010, 111, 1001.

Learning Outcome:

Understand pattern matching using NFA, useful in regex engines.

SOLUTION:

Language

Accept every string containing **101** anywhere.

States

- q_0 = Start
 - q_1 = Seen 1
 - q_2 = Seen 10
 - q_3 = Seen 101 (Final)
-

Transition Table

State	0	1
$\rightarrow q_0$	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\{q_2\}$	$\{q_1\}$
q_2	$\{q_0\}$	$\{q_3\}$
$*q_3$	$\{q_3\}$	$\{q_3\}$

Explanation

Whenever a **1** appears, the NFA starts checking for **101**.

If the guess is wrong,

it simply tries again from another position.

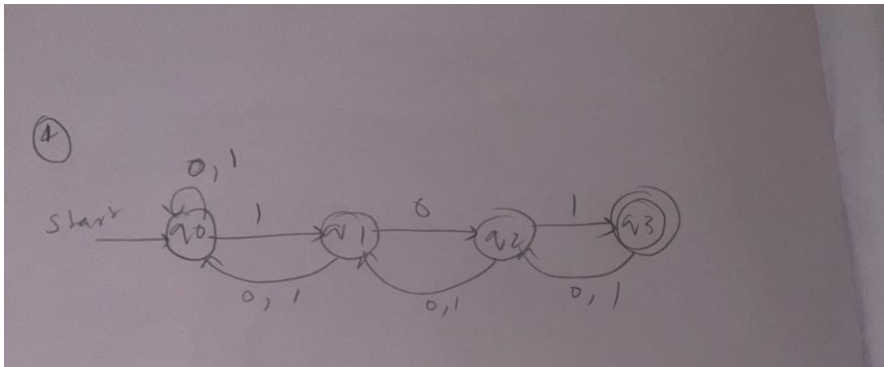
This is why NFAs are useful for substring searching.

Testing

String	Accepted?	Reason
101	✓	Contains 101
01010	✓	Contains 101
111	✗	No 101
1001	✗	No 101

Learning Outcome

Understand how NFAs perform pattern matching similar to regular expression engines.



Problem 5: Design NFA First, Then Convert to DFA

Problem:

Design an NFA for the language:

All strings over $\{0,1\}$ that contain the substring 01

Then convert that NFA to an equivalent DFA.

Student Task:

- Draw the NFA.
- Apply subset construction.
- Compare number of states in NFA vs DFA.
- Comment: "Which is simpler to design? Why?"

Learning Outcome:

Shows why NFAs are easier to design but DFAs are easier to implement.

SOLUTION:

Language

Strings containing substring **01**

Step 1: NFA

States

- q_0
- q_1
- q_2 (Final)

Transition Table

State	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\{q_2\}$	$\{q_2\}$

Step 2: DFA (Subset Construction)

DFA States

- $A = \{q_0\}$
- $B = \{q_0, q_1\}$

- $C = \{q_0, q_2\}$

DFA Transition Table

DFA State	0	1
$\rightarrow A = \{q_0\}$	B	A
$B = \{q_0, q_1\}$	B	C
$*C = \{q_0, q_2\}$	C	C

Comparison

Feature	NFA	DFA
States	3	3
Multiple transitions	Yes	No
ϵ -transitions	Allowed	Not Allowed
Design	Easier	Harder
Implementation	Difficult	Easier

Comment

NFA is simpler to design because it can use **nondeterminism** to guess the correct path. **DFA is easier to implement** because for every state and input symbol there is exactly one transition, making execution straightforward and efficient.

